

6. Заглянем базе внутрь

- Расширение pageinspect

```
CREATE EXTENSION pageinspect;
```

Делаем небольшую таблицу для опытов

```
CREATE TABLE test (  
    id    int NOT NULL,  
    y     int,  
    z     text  
);  
CREATE INDEX test_i ON test(id);  
INSERT INTO test VALUES (1,2,'Шишка');  
INSERT INTO test VALUES (3);
```



Как заглянуть в страницу базы

- Используем pageinspect

```
SELECT get_raw_page('test', 0);
```

Что-то вначале, что-то в конце, в середине много нулей.

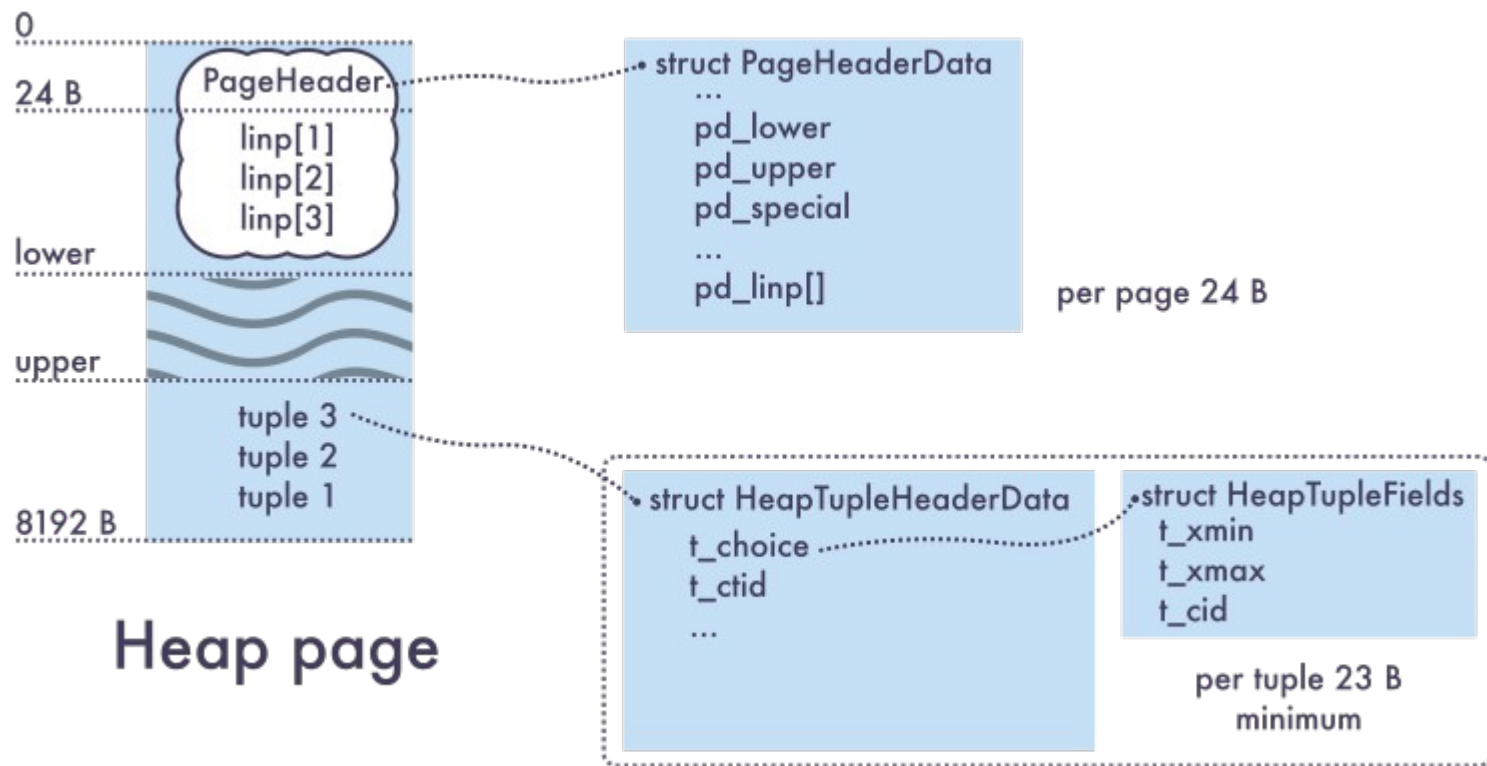
- Сколько будет `SELECT length(get_raw_page('test', 0));` ?

Правильно, 8k

Какой возвращаемый тип `get_raw_page` ?



Структура страницы



- Из статьи Максима Орлова на Habr



Ещё немного о терминологии

- MVCC — Multi Version Concurrency Control (многоверсионное управление одновременным доступом)
- Запись (строка) БД — видимая пользователю строка таблицы. При её изменении порождаются версии строки (записи).
- Кортеж (tuple) - версия записи. Их у одной записи может быть много в результате MVCC — как на одной странице, так и на разных
- HOT - «heap only tuple» - технология для сокращения избыточной записи версий записей в индексы (Write Amplification)

Заголовок страницы

- Используем pageinspect

```
SELECT * FROM page_header(get_raw_page('test', 0));
```

Заголовок страницы

- Используем pageinspect

```
SELECT * FROM page_header(get_raw_page('test', 0));
```

lsn	15/C70EFAE8	-- позиция в WAL (последняя для страницы)
checksum	0	-- если контрольные суммы включены
flags	0	-- битовые флаги
lower	32	-- нижняя граница свободного места страницы
upper	8122	-- верхняя граница свободного места страницы
special	8192	-- смещение специального места страницы
pagesize	8192	-- размер страницы (хотя это константа)
version	4	-- версия лейаута страницы
prune_xid	0	-- старейший xid записей, которые можно зачистить

Кроме заголовка

- Используем pageinspect

```
SELECT * FROM heap_page_items(get_raw_page('test', 0));
```

Кроме заголовка

- Используем pageinspect

```
SELECT * FROM heap_page_items (get_raw_page('test', 0));
```

lp	1	-- номер п.п. (lp = Line Pointer)
lp_off	8144	-- смещение от начала страницы
lp_flags	1	-- 2 бита (unused, normal, HOT redirect, dead)
lp_len	43	-- длина
t_xmin	202291	-- XID, начиная с которого видно
t_xmax	0	-- XID, начиная с которого не видно (или LOCK)
t_field3	0	-- хитрое служебное поле
t_ctid	(0, 1)	-- «физический идентификатор» кортежа
t_infomask2	3	-- хитрые биты
t_infomask	2306	-- ещё хитрые биты
t_hoff	24	-- длина заголовок кортежа
t_bits		-- битмап NULL'ов
t_oid		
t_data	\x01000000002000000017d0a8d0b8d188d0bad0b0	

Что там за хитрые биты

- Используем pageinspect

```
SELECT lp,  
       heap_tuple_infomask_flags(t_infomask, t_infomask2)  
FROM heap_page_items(get_raw_page('test', 0)) ;
```

- HASVARWIDTH — есть поля переменной длины
- HASNULL — есть NULL
- XMIN_COMMITTED
- XMAX_COMMITTED -- «hint bits» пишутся не сразу, для оптимизации выборки
- и др

Как разобрать кортеж на отдельные атрибуты?

```
tuple_data_split (...)  
heap_page_item_attrs (...)
```

Получается...

```
SELECT lp, t_xmin, t_xmax,  
       heap_tuple_infomask_flags(t_infomask, t_infomask2),  
       t_attrs  
FROM heap_page_item_attrs(get_raw_page('test', 0), 'test') ;  
  
hint-биты проставляет VACUUM (проверьте)
```

- Если это HINT, где первичная информация?
- CLOG

```
xid => статус транзакции (IN_PROGRESS, COMMITTED, ABORTED)  
txid_status (xid)
```

Сделаем UPDATE

```
BEGIN;  
SELECT lp, t_xmin, t_xmax,  
       heap_tuple_infomask_flags(t_infomask, t_infomask2),  
       t_attrs  
FROM heap_page_item_attrs(get_raw_page('test', 0), 'test') ;  
UPDATE test SET y = 5 WHERE id = 3;
```

```
SELECT ...
```

```
ROLLBACK;
```

```
SELECT ....
```

что-то меняется ? Увидим, обсудим.

VACUUM ?

Сделаем SELECT FOR UPDATE

```
BEGIN;  
SELECT * FOR UPDATE FROM test
```

что-то меняется ? Увидим, обсудим.
VACUUM ?

```
CREATE EXTENSION pgrowlocks;  
SELECT * pgrowlocks('test');
```

интермедия

А что не так с SERIALIZABLE

Предикатные блокировки
как они устроены?

Рекурсивный SQL

- Common table expressions

```
WITH    tab1 AS (SELECT .....),  
        tab2 AS (SELECT ...)  
SELECT * FROM tab1, tab2
```

```
WITH RECURSIVE  
    tab1 AS (  
        SELECT first_part  
        UNION ALL  
        SELECT second_part  
    )  
SELECT * FROM tab;
```

Рекурсивный пример

```
CREATE TABLE node (  
    id      int PRIMARY KEY,  
    parent int REFERENCES node(id),  
    title   text  
);
```

```
WITH RECURSIVE tree AS (  
    SELECT id, title, 0 AS level, ARRAY[id] AS path  
    FROM node  
    WHERE parent IS NULL  
    UNION  
    SELECT node.id, node.title, tree.level + 1, path || ARRAY[node.id]  
    FROM tree JOIN node ON tree.id = node.parent  
    WHERE level < 3 -- «на всякий аварийный случай»  
)  
SELECT * FROM tree ORDER BY path;
```

Задачи

- Какой самый длинный неповторяющийся аэропорты маршрут из Каламы (CJS)?
- А вообще в базе ?

Разные индексы

- Понимание индексов
- B-дерево
- Hash
- GiST
- *GIN*
- *BRIN*
- Индексы и производительность

- Сравнительно компактная структура для быстрого поиска



Все знают

- Бинарное дерево
- Хеш-таблица

Каковы плюсы и минусы каждого?

Сравним

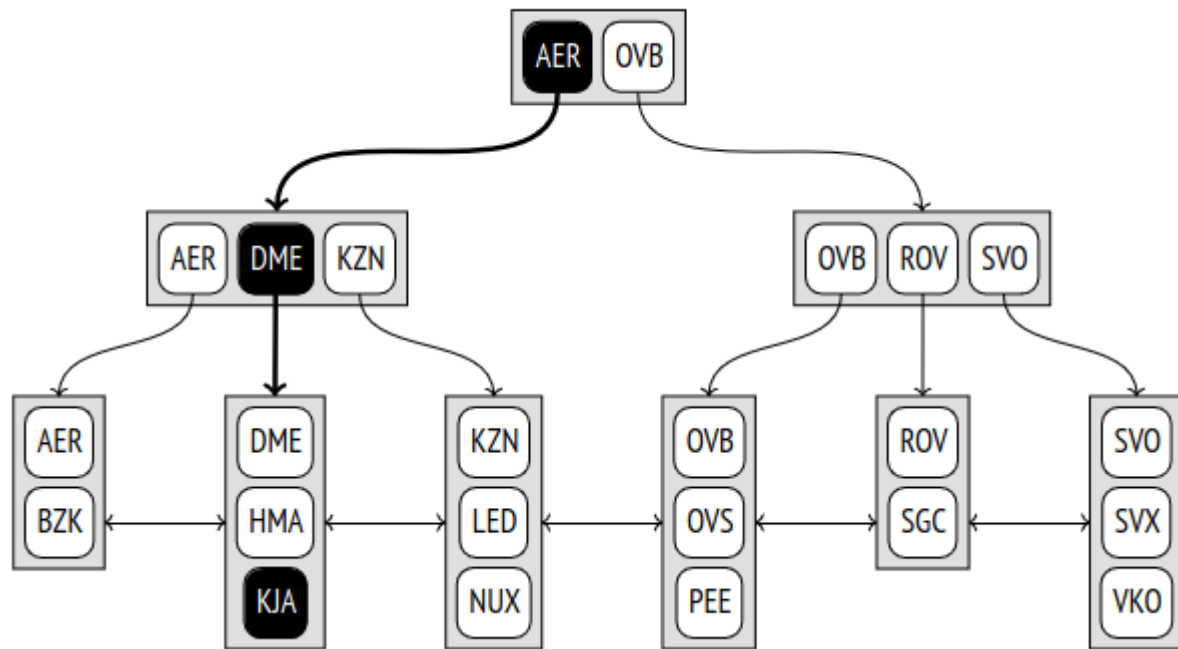


Г.М. Адельсон-Вельский

Хорошо	Худо
Бинарное дерево	
Логарифмический поиск и вставка	Это так, только если дерево сбалансировано. AVL; RB Много случайных доступов к памяти
Дерево большей арности	
Меньше походов в память. Хорошо для двухуровневой памяти.	Сложнее алгоритмы. В-дерево Блокировки коренных узлов
Хеш	
Быстрый поиск и вставка	(если нет перестройки хеш-таблицы) не задает порядок — нельзя сортировать Нельзя UNIQUE и multicolumn

В-дерево

- В каждом узле — список ключей
- Сбалансировано
- Отсортировано
- Данные — внизу
- При вставке — м.б. split (расщепление) узлов
- Узел должен уместиться на страницу (8k)



Из книги Е. Рогова «PostgreSQL изнутри»

Польза от индекса

\timing on

```
INSERT INTO student  
  SELECT generate_series (1,1000000), random_string(10);
```

```
SELECT * FROM student WHERE name = 'аддваебге';
```

```
CREATE INDEX student_name ON student (name) ;
```

```
INSERT INTO student  
  SELECT generate_series (1,1000000), random_string(10);
```

```
SELECT * FROM student WHERE name = 'аддваебге';
```

Сравним и сделаем выводы

ТОНКОСТИ

```
SELECT * FROM student WHERE name LIKE 'аддвае%';
```

```
CREATE INDEX student_name1  
ON student ( name text_pattern_ops ) ;
```

```
SELECT * FROM student WHERE name LIKE 'аддвае%';
```

Сравним и сделаем выводы

HASH



Из книги Е. Рогова «PostgreSQL изнутри»

Польза от индекса:

- сомнительная
- меньше, чем B-Tree
- другое поведение при UPDATE



GiST

B-дерево привязано к операции сравнения.

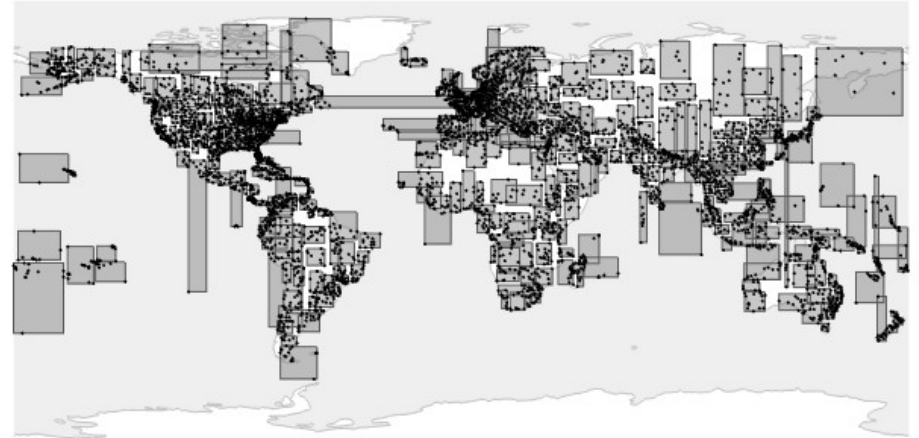
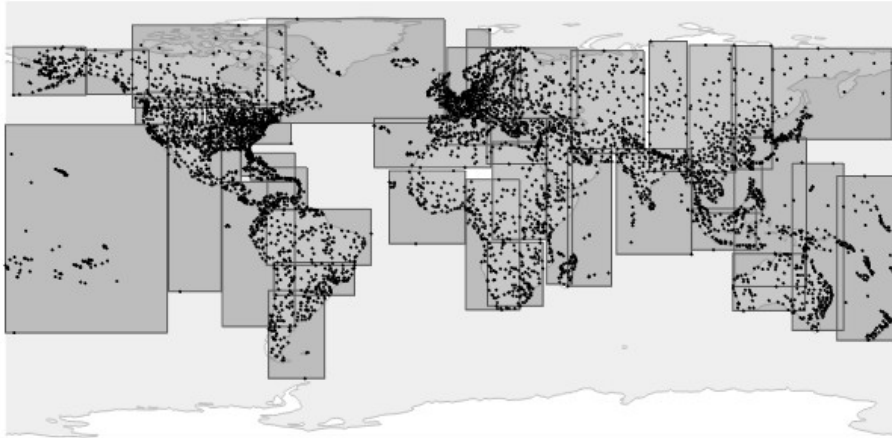
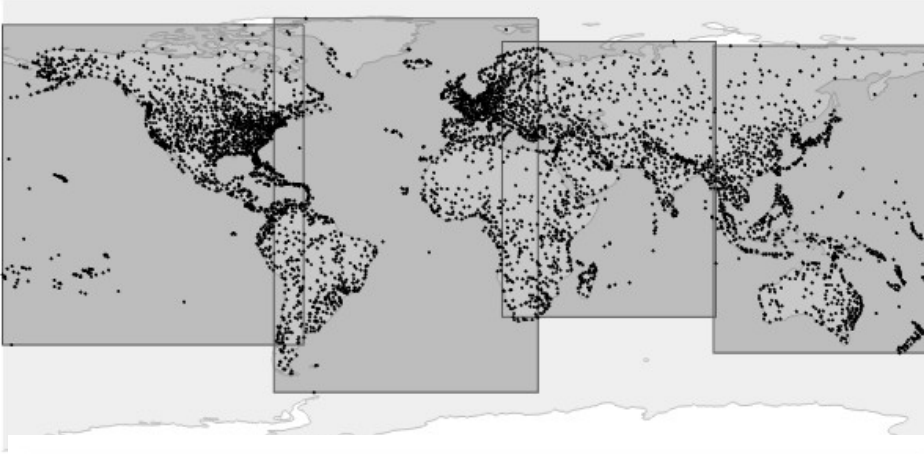
GiST — это более высокий уровень абстракции. Поэтому можно индексировать «несравнимое». Например, точки пространства и множества для поиска по вхождению.

- 1) входит ли?
- 2) как расщеплять.

Если это известно (можно создать «класс операторов»), можно применять GiST.

R-дерево

«Точка внутри бокса»



Точки

```
SELECT * FROM airports WHERE coordinates <@ '((36,55),(38,57))'::box;
```

EXPLAIN ?

```
CREATE TABLE points AS  
SELECT point(random()*100, random()*100)  
FROM generate_series(1,100000);
```

```
EXPLAIN SELECT * FROM points WHERE point <@ '((36,55),(38,57))'::box;
```

А если уменьшить box ?

Делаем выводы.

КОНЕЦ

К следующему разу:

Порешать задачу (Д3)